

Nozioni di Teoria — ICC / LAAI Module 3

Complessità computazionale, decidibilità, NP-completezza, PAC-learning

Contents

1. Notazione asintotica (O, Ω, Θ)	2
1.1 Definizioni	2
1.2 Metodo pratico: il rapporto	2
1.3 Trucchi ricorrenti	2
2. Codificabilità come stringhe binarie	3
3. Turing Machine: proprietà generali	3
3.1 Schema per costruire una TM “a scansione singola”	3
4. Decidibilità e Teorema di Rice	4
4.1 Concetti base	4
4.2 Teorema di Rice	4
4.3 Schema di riduzione per dimostrare indecidibilità	4
5. Gerarchia delle classi di complessità	4
5.1 Inclusioni dimostrate (certe)	4
5.2 Separazioni dimostrate (certe)	5
5.3 Relazioni congetture ma NON dimostrate	5
5.4 coNP	5
6. Riduzioni polinomiali e NP-completezza	5
6.1 Definizioni	5
6.2 Schema fisso per dimostrare $L \in NP$	6
6.3 Schema fisso per dimostrare NP-completezza	6
6.4 Relazioni strutturali utili tra VERTEXCOVER, INDSET, CLIQUE	6
7. Pattern ricorrente: “fissare un parametro a costante” $\Rightarrow P$	7
8. FP e FEXP (classi funzionali)	8
8.1 Schema per dimostrare $f \in FP$	8
8.2 Esempio di problema in FEXP (ma non noto in FP)	8
9. PAC-Learning e VC-dimension	8
9.1 Definizioni chiave	8
9.2 VC-dimension: cos’è e come si dimostra	8

9.3 VC-dimension note (esempi ricorrenti)	9
9.4 Congiunzioni di letterali: perché sono efficiently PAC-learnable	10
9.5 Disgiunzioni di letterali: dualità con le congiunzioni	10
9.6 Amico con $SAT \subseteq L$ decidibile in tempo polinomiale (pattern “sovrainsieme”) 10	
9.7 Amico con problema EXP-completo trovato anche in NP	10

10. Checklist finale per rispondere ai quiz 11

1. Notazione asintotica (O , Ω , Θ)

1.1 Definizioni

- $f \in O(g)$ se esistono $c > 0, n_0$ tali che $f(n) \leq c \cdot g(n)$ per ogni $n \geq n_0$ (f **non cresce più velocemente** di g).
- $f \in \Omega(g)$ se esistono $c > 0, n_0$ tali che $f(n) \geq c \cdot g(n)$ per ogni $n \geq n_0$ (f **cresce almeno quanto** g).
- $f \in \Theta(g)$ se $f \in O(g)$ e $f \in \Omega(g)$ (f e g **crescono allo stesso ordine**).

1.2 Metodo pratico: il rapporto

Calcola $\lim_{n \rightarrow \infty} f(n)/g(n)$:

Limite	Conclusione
0	$f \in O(g), f \notin \Omega(g)$
costante $c > 0$	$f \in \Theta(g)$
∞	$f \in \Omega(g), f \notin O(g)$

1.3 Trucchi ricorrenti

- **Riscrivi per far combaciare gli esponenti.** Es: $n \cdot n^n = n^{n+1}$. Se dopo la riscrittura le costanti moltiplicative sono le uniche differenze, hai Θ .
- **Esponenti “esponenziali nell’esponente” dominano tutto il resto.** Confrontando n^{2^n} e n^{3^n} : guarda **come cresce l’esponente stesso** (2^n vs 3^n), non le costanti o i fattori log davanti — questi ultimi diventano irrilevanti. 3^n cresce più in fretta di 2^n , quindi n^{3^n} domina n^{2^n} indipendentemente da tutto il resto.
- n^{cn} **batte sempre** b^n (esponenziale a base costante), per qualunque costante b fissata, perché la base n^c cresce essa stessa con n .
- **Fattoriali: usa Stirling.** $n! \approx n^n \cdot e^{-n} \cdot \sqrt{2\pi n}$.
 - Utile per confrontare $c^n \cdot n!$ con n^n : il rapporto si riduce a $(c/e)^n \cdot \text{poly}(n)$. Se $c > e$, il prodotto cresce più di n^n (quindi Ω , non Θ); se $c < e$ decresce; solo per $c = e$ (caso limite) il confronto diventa più delicato.
 - $\log(n!) \approx n \log n - n + O(\log n)$, quindi $\log(n!) \in \Theta(n \log n)$ — risultato molto usato (es. limite inferiore per l’ordinamento a confronto, $\Omega(n \log n)$).

2. Codificabilità come stringhe binarie

È **codificabile facilmente** (rappresentazione finita ed esatta): - Numeri naturali, interi, razionali (come coppie numeratore/denominatore). - Alfabeti finiti di qualsiasi dimensione **costante** (anche grande, es. 10^4): bastano $\lceil \log_2(k) \rceil$ bit per simbolo. - Grafi finiti, sequenze/vettori finiti di oggetti già codificabili (es. sequenze finite di naturali o di razionali).

NON è codificabile facilmente: - Numeri reali o complessi **generici** (richiedono infinite cifre in generale). - Sequenze **finite** di reali/complessi arbitrari (basta un solo reale generico a rompere la codificabilità). - Sequenze **infinite** di qualsiasi cosa (bit, reali...) — una stringa ha comunque solo una quantità numerabile di simboli, ma un oggetto infinito arbitrario porta informazione non gestibile in modo finito.

3. Turing Machine: proprietà generali

- Hanno un numero **finito di stati**, ma **configurazioni infinite** (una configurazione include stato + contenuto del nastro + posizione testina; il nastro non è limitato a priori).
- Possono avere **tanti nastri quanti servono** (il numero di nastri è deciso in fase di progettazione, non cambia la classe P/NP di un problema — cambia solo il fattore polinomiale/costante di complessità).
- **Non terminano sempre:** esistono TM che vanno in loop infinito su certi input (base dell'Halting Problem).
- Un problema **multi-nastro** si può sempre simulare con **singolo nastro** con un overhead **polinomiale** (spesso quadratico) — non cambia la classe di complessità (P resta P), ma può cambiare l'esponente esatto (es. palindromo: $O(n)$ con 2+ nastri, $O(n^2)$ con nastro singolo, perché la testina deve fare avanti-indietro fisicamente).

3.1 Schema per costruire una TM “a scansione singola”

Per molti linguaggi del tipo $L = \{w \mid \text{proprietà locale/di conteggio su } w\}$: 1. Identifica l'informazione minima necessaria da “ricordare” mentre scorri il nastro da sinistra a destra (es. parità di 0/1 vista finora, ultimo simbolo, quanto di un pattern hai già visto). 2. Codifica questa informazione **nello stato** (non serve altro nastro se l'informazione è di dimensione costante/bounded). 3. Ad ogni simbolo letto, aggiorna lo stato di conseguenza. 4. Accetta/rifiuta solo quando leggi il simbolo di **blank** (fine stringa) — mai a metà computazione. 5. Se serve confrontare due conteggi indipendenti (es. “pari 0 e dispari 1”), spesso serve più di un nastro di appoggio, ma la complessità resta $O(n)$ se fatto bene.

Complessità tipica: $O(n)$ con scansione singola; $O(n^2)$ se serve confrontare simboli lontani con nastro singolo (es. palindromo).

4. Decidibilità e Teorema di Rice

4.1 Concetti base

- **Decidibile:** esiste una TM che per **ogni** input termina e dà la risposta corretta.
- **Indecidibile:** non esiste nessun algoritmo che risolva il problema per tutti gli input in tempo finito. (*Decidibile e indecidibile sono concetti opposti/mutuamente esclusivi, non equivalenti — attenzione a non confonderli.*)
- **Semi-decidibile / riconoscibile:** esiste una TM che accetta esattamente le stringhe nel linguaggio, ma su input **non** nel linguaggio può anche non terminare (loop). Se sia L che $\bar{L}$ (complemento) sono semi-decidibili, allora L è decidibile.

4.2 Teorema di Rice

Ogni proprietà non banale del linguaggio riconosciuto da una TM è indecidibile.

- “Non banale” = esiste almeno una TM che soddisfa la proprietà e almeno una che non la soddisfa.
- “Del linguaggio riconosciuto” = proprietà **semantica** (sul comportamento/output), non sintattica.
- **Eccezioni (decidibili):** proprietà puramente **sintattiche** leggibili direttamente dalla descrizione della macchina senza eseguirla (es. “M ha al più 10 stati”, “la descrizione di M contiene il simbolo ‘a’”).

Esempi di problemi indecidibili per Rice: HALT (M termina su x ?), HALT_0 (M termina su ϵ ?), EMPTY ($L(M) = \emptyset$?), EQ ($L(M_1) = L(M_2)$?), “M accetta 00101?”, “M rifiuta 00101?” — qualunque domanda sul comportamento di una TM generica su input specifici o generali.

4.3 Schema di riduzione per dimostrare indecidibilità

Per mostrare che un problema X è indecidibile, riduci **HALT** (o un altro problema noto indecidibile) a X : 1. Assumi per assurdo che esista un decisore D_X per X . 2. Dato un’istanza $\langle M, x \rangle$ di HALT , costruisci (in tempo qualsiasi, non serve essere polinomiale qui) una nuova macchina M' che “incapsula” il comportamento di M su x in modo che $M' \in X \iff M$ termina su x . 3. Usa $D_X(M')$ per decidere HALT — contraddizione, perché HALT è indecidibile.

Esempio concreto (HALT_0): dato $\langle M, x \rangle$, costruisci M' che ignora il proprio input, scrive x sul nastro, e simula M su x . Allora M' termina su $\epsilon \iff M$ termina su x .

5. Gerarchia delle classi di complessità

5.1 Inclusioni dimostrate (certe)

$$P \subseteq NP \subseteq PSPACE \subseteq EXP$$

- $P \subseteq NP$: ogni problema in P ha banalmente un verificatore polinomiale (certificato vuoto).

- $NP \subseteq PSPACE$: si può provare ogni certificato riusando lo spazio.
- $PSPACE \subseteq EXP$: spazio polinomiale $\Rightarrow$ al più esponenzialmente tante configurazioni distinte.

5.2 Separazioni dimostrate (certe)

- $P \subsetneq EXP$ (quindi $P \neq EXP$) — **Time Hierarchy Theorem**: più tempo (esponenziale vs polinomiale) permette di risolvere strettamente più problemi. Dimostrazione costruttiva, non dipende da congetture aperte.
- $PSPACE \subsetneq EXPSPACE$ — analogo, **Space Hierarchy Theorem**.

5.3 Relazioni congetturate ma NON dimostrate

- $P \neq NP$ (il problema aperto per eccellenza).
- $NP \neq PSPACE$, $P \neq PSPACE$.
- $NP \neq coNP$.
- Qualunque disuguaglianza che coinvolga **NPC** (classe dei problemi NP-completi), perché dipende in cascata da P vs NP : es. " $P \neq NPC$ " sembra ovvia ma **non è nota con certezza**, dato che presuppone $P \neq NP$.

Criterio pratico per riconoscere queste domande nei quiz: se la relazione coinvolge P , NP , $PSPACE$, NP tra loro (senza $EXP/EXPSPACE$) $\rightarrow$ quasi sempre aperta/congetturata. Se coinvolge un salto verso EXP o $EXPSPACE$ tramite hierarchy theorem $\rightarrow$ dimostrata.

5.4 coNP

- $coNP$ = classe dei problemi il cui **complemento** è in NP .
- Se L è **NP-completo**, allora $\bar{L}$ (il complemento, stesso tipo di istanze ma risposta invertita) è **coNP-completo**. (Regola specifica di NP — non generalizzare ad altre classi senza verificare.)
- Se $L \in P$, allora anche $\bar{L} \in P$ — P è chiusa per complemento (basta invertire l'output dello stesso algoritmo). Quindi il complemento di un problema in P resta in P , non genera una nuova classe.
- Esempio: 3SAT (formule soddisfacibili) è NP-completo; COMPLEMENT-3SAT (formule insoddisfacibili) è coNP-completo. Si congettura (non dimostrato) che $NP \neq coNP$, cioè che non esista un certificato breve per dimostrare l'**insoddisfacibilità** di una formula generica.

6. Riduzioni polinomiali e NP-completezza

6.1 Definizioni

- $L \leq_p H$ (L si riduce polinomialmente a H) se esiste una funzione f calcolabile in tempo polinomiale tale che $x \in L \iff f(x) \in H$.
- Se $L \leq_p H$, allora H è **almeno difficile quanto** L .
- H è **NP-hard** se ogni problema in NP si riduce polinomialmente a H .

- H è **NP-completo** se $H \in NP$ e H è NP-hard.

6.2 Schema fisso per dimostrare $L \in NP$

1. **Certificato:** definisci un oggetto C (di dimensione **polinomiale** nell'input) che "testimonia" la risposta sì.
2. **Verificatore:** pseudocodice/TM che, dati input e certificato, controlla in tempo polinomiale se il certificato è valido.
3. Dimostra che il verificatore è polinomiale controllando:
 - input codificabile in binario;
 - **numero di istruzioni** polinomiale nella dimensione dell'input;
 - **dimensione dei risultati intermedi** polinomiale;
 - ogni istruzione elementare (somme, confronti, assegnazioni, cicli semplici) eseguibile in tempo polinomiale.
4. Conclusione: $L \in NP$.

Trucco spesso utile: non serve includere nel certificato informazioni ricalcolabili facilmente dal verificatore stesso (es. per "n è prodotto di due primi $p^a q^b$ ", il certificato può essere solo (p, q) , non anche gli esponenti a, b — questi si ricalcolano dividendo ripetutamente).

6.3 Schema fisso per dimostrare NP-completezza

1. Dimostra $L \in NP$ (schema sopra).
2. Scegli un problema H **già noto NP-completo** (SAT, 3SAT, CLIQUE, INDSET, VERTEXCOVER, SUBSETSUM, HAMPATH...).
3. Costruisci una riduzione polinomiale $H \leq_p L$ — una funzione f calcolabile in tempo polinomiale che trasforma istanze di H in istanze equivalenti di L (stessa risposta sì/no).
4. Concludi: L è NP-hard (dal passo 3) e $L \in NP$ (dal passo 1) $\Rightarrow L$ è NP-completo.

Attenzione: una riduzione va sempre dimostrata nelle **due direzioni** (se... allora, e viceversa) — un errore comune è dare solo la riduzione informale in un verso, come segnalato spesso dal tutor nei tuoi compiti.

6.4 Relazioni strutturali utili tra VERTEXCOVER, INDSET, CLIQUE

Dato un grafo $G = (V, E)$ e $W \subseteq V$:

$$W \text{ è vertex cover di } G \iff V \setminus W \text{ è independent set di } G$$

Dimostrazione: W copre ogni arco (ogni arco ha un estremo in W) $\iff$ nessun arco ha entrambi gli estremi fuori da W $\iff V \setminus W$ non contiene archi $\iff V \setminus W$ è independent set. Vale per **ogni** grafo, non solo bipartiti.

Conseguenza: $(G, k) \in INDSET \iff (G, |V| - k) \in VERTEXCOVER$ — riduzione immediata in entrambe le direzioni, utile per trasferire NP-completezza tra i due problemi.

Analogamente, un independent set di G è una **clique** nel grafo **complementare** $\bar{G}$ (dove gli archi sono esattamente le non-adiacenze di G) — questo collega INDSET a CLIQUE.

7. Pattern ricorrente: “fissare un parametro a costante” $\Rightarrow$ **P**

Molti problemi NP-completi hanno varianti **in P** quando un parametro che normalmente è **libero nell’input** viene **fissato a una costante** (indipendente dalla dimensione dell’istanza), oppure quando la struttura dell’istanza viene ristretta in modo da eliminare la scelta combinatoria.

Esempi visti:

Problema NP-completo	Variante in P	Perché
CLIQUE (k libero)	k -CLIQUE con k costante (es. THREECLIQUE, 5-CLIQUE)	Provare tutte le $\binom{ V }{k}$ combinazioni è $O(V ^k)$: polinomiale se k è costante
k -SAT generico o SAT	3SATVAR (al più 4 variabili)	Con m variabili costanti, ci sono al più 2^m assegnamenti: costante, si provano tutti
INDEPENDENT SET	ONEINDSET (ogni nodo in al più un arco $\Rightarrow$ grafo = matching)	Struttura ristretta elimina la scelta combinatoria: greedy funziona
HAMILTONIAN PATH	HAMPATHVAR (grado massimo del grafo ≤ 2)	Il grafo è unione di cammini/cicli: il cammino Hamiltoniano (se esiste) è quasi obbligato dalla struttura
KNAPSACK	VARKNAPSACK (tutti i pesi = 1)	Il vincolo di peso diventa un vincolo di conteggio: greedy (prendi i valori più alti) è ottimo
“G contiene una clique grande $ V $ ”	sempre in P	Equivale a “G è completo?”: basta controllare tutte le coppie, $O(V ^2)$
Verificare se F (3CNF) è insoddisfacibile con variabili bounded	in P (anzi $O(1)$ nel numero di variabili)	Stesso principio di 3SATVAR

Regola pratica generale: quando un esercizio introduce una variante di un problema NP-completo con un parametro “strano” fissato o una struttura ristretta, il

sospetto principale deve essere che il problema **collassi in P** — cerca sempre prima un algoritmo diretto/greedy prima di lanciarti in una riduzione NP-hard.

8. FP e FEXP (classi funzionali)

- **FP**: funzioni calcolabili in tempo polinomiale (analogo funzionale di P).
- **FEXP**: funzioni calcolabili in tempo esponenziale.
- Nota: $L \in P \Rightarrow$ la funzione caratteristica di L è in FP; ma il viceversa **non vale** in generale (una funzione può essere in FP senza che il linguaggio “associato” sia in P, dipende da cosa si intende per linguaggio associato).

8.1 Schema per dimostrare $f \in FP$

Stesso schema del verificatore NP (punto 6.2, passi 3), applicato direttamente all'algoritmo che calcola f (non a un verificatore di certificato): - input codificabile in binario; - numero di istruzioni polinomiale; - dimensione dei risultati intermedi polinomiale; - ogni istruzione elementare in tempo polinomiale.

Esempi classici in FP: prodotto scalare di due liste di razionali; verificare se una stringa è sottostringa di un'altra; $MIRROR(w) = w \cdot w^R$; verificare se un array è ordinato (SORTED-CHECK); trovare min e max di una lista; ordinare un array (SORT).

8.2 Esempio di problema in FEXP (ma non noto in FP)

FINDSET: dato un grafo, trovare un sottoinsieme massimale di nodi a due a due non adiacenti — risolvibile enumerando tutti i $2^{|V|}$ sottoinsiemi, quindi in FEXP; nessun algoritmo polinomiale noto (collegato a INDSET/NP-completezza).

9. PAC-Learning e VC-dimension

9.1 Definizioni chiave

- Una classe di concetti è **PAC-learnable** se esiste un algoritmo che, con probabilità almeno $1 - \delta$, produce un'ipotesi con errore al più ϵ , usando un numero di esempi **polinomiale** in $1/\epsilon$, $1/\delta$ (e nella dimensione della rappresentazione del concetto).
- È **efficiently PAC-learnable** se in più l'algoritmo gira in **tempo polinomiale**.
- **Teorema fondamentale**: una classe di concetti è PAC-learnable **se e solo se** ha **VC-dimension finita**.

9.2 VC-dimension: cos'è e come si dimostra

La VC-dimension di una classe C è la dimensione del più grande insieme di punti che C può **shatterare** (cioè per ogni possibile etichettatura $\pm$ dei punti, esiste un concetto in C che la realizza esattamente).

Per dimostrare $VCdim(C) = d$ servono due parti: 1. **Lower bound** ($VCdim \geq d$): esibisci **un insieme specifico** di d punti e mostra che **ogni** etichettatura possibile

(2^d in totale) è realizzabile da qualche concetto in C . 2. **Upper bound** ($VCdim < d + 1$): mostra che **nessun** insieme di $d + 1$ punti può essere shatterato (di solito per assurdo: assumi che esista, e trova un’etichettatura specifica che nessun concetto in C può realizzare).

9.3 VC-dimension note (esempi ricorrenti)

Classe di concetti	VC-dimension	Efficiently PAC-learnable?
Intervalli chiusi in $\mathbb{R}$	2	Sì
Rettangoli assiali in $\mathbb{R}^2$	4	Sì
Quadrati assiali in $\mathbb{R}^2$	3	Sì
Cerchi/dischi in $\mathbb{R}^2$	3	Sì
Semipiani in $\mathbb{R}^2$	3	Sì
Congiunzioni di letterali (n variabili)	— (analisi diversa, via consistenza)	Sì
Disgiunzioni di letterali	— (duale delle congiunzioni)	Sì (per simmetria, vedi 9.5)
3-term DNF	—	No , se $RP \neq NP$ (NP-hard da apprendere)

Nota generale: forme geometriche “semplici” con pochi parametri hanno quasi sempre VC-dimension piccola e costante $\Rightarrow$ efficiently PAC-learnable con algoritmo greedy (es. “il rettangolo/quadrato/cerchio più piccolo consistente con gli esempi positivi”). Le eccezioni difficili nascono da strutture **combinatorie** che codificano problemi NP-hard (es. 3-term DNF, dimostrato via riduzione da 3-colorabilità di grafi).

9.3.1 Attenzione alle risposte “con motivazione” (VC-dimension finita/infinita esplicitata)

Alcune varianti del quiz non chiedono solo “è PAC-learnable?” ma legano esplicitamente la conclusione alla **causa strutturale**, con opzioni tipo:

- “PAC learnable, but not efficiently so, **having infinite VC-dimension.**” — quasi sempre **falsa** per forme geometriche semplici (quadrati, rettangoli, cerchi, semipiani...): la VC-dimension è **finita** (piccola: 2, 3 o 4), non infinita. Se la VC-dimension fosse davvero infinita, il concetto **non sarebbe nemmeno PAC-learnable** (per il teorema fondamentale) — quindi questa frase è internamente incoerente (“learnable” + “VC-dimension infinita” non possono coesistere).
- “Efficiently PAC learnable, **having finite VC-dimension.**” — questa è di solito la risposta **corretta e più forte**: unisce correttamente causa (VC-dim finita) ed effetto (efficiently learnable).
- “PAC learnable” (senza altra qualifica) — è **vera ma più debole/generica**: non sbagliata, ma se è disponibile un’opzione più specifica e corretta (“efficiently... finite VC-dim”), quella va scelta **in aggiunta**, non al posto dell’altra (spesso vanno selezionate entrambe se il quiz è multi-risposta).

Regola pratica: quando un'opzione abbina “PAC learnable” (o “not learnable”) a una giustificazione esplicita sulla VC-dimension (finita/infinita), verifica **prima la VC-dimension reale** della classe di concetti (Sezione 9.3) e poi controlla che l'abbinamento $\text{learnability} \leftrightarrow \text{VC-dimension}$ sia logicamente coerente ($\text{VC-dim finita} \iff \text{learnable}$; $\text{VC-dim infinita} \iff \text{non learnable}$ — mai il contrario).

9.4 Congiunzioni di letterali: perché sono efficiently PAC-learnable

Formule del tipo $x_1 \wedge \neg x_2 \wedge x_5 \wedge \dots$ (AND di variabili, eventualmente negate).

Algoritmo: parti assumendo che tutti i $2n$ letterali possibili facciano parte della congiunzione ipotesi. Per ogni esempio **positivo** ricevuto, elimina dall'ipotesi ogni letterale che risulta **falso** in quell'esempio. Alla fine restano solo i letterali consistenti con tutti i positivi visti.

- Tempo polinomiale (un passaggio per esempio).
- Si dimostra che il numero di esempi necessario per la garanzia PAC è polinomiale.
- Sono “più facili” da apprendere delle CNF generiche, che includono come caso particolare strutture NP-hard (3-term DNF).

9.5 Disgiunzioni di letterali: dualità con le congiunzioni

Per De Morgan: $\neg(x_1 \vee \neg x_3 \vee x_5) = \neg x_1 \wedge x_3 \wedge \neg x_5$.

La negazione di una disgiunzione di letterali è una congiunzione di letterali (segni invertiti). Quindi si riduce l'apprendimento delle disgiunzioni a quello delle congiunzioni **invertendo le etichette** degli esempi (positivo $\leftrightarrow$ negativo) — trasformazione gratuita, stesso algoritmo, stessa efficienza. Le due classi hanno quindi **esattamente la stessa difficoltà** di apprendimento.

9.6 Amico con $\text{SAT} \subseteq L$ decidibile in tempo polinomiale (pattern “sovrainsieme”)

Se ti dicono che $L \supseteq \text{SAT}$ (o un altro problema NP-hard) ed L è in P, **non puoi concludere nulla su P vs NP**: L potrebbe essere un sovrainsieme “banale” (es. tutte le stringhe), che ovviamente contiene SAT ma non dice nulla sulla difficoltà di SAT stesso. Serve invece parlare di **SAT direttamente** (o un problema equivalente per riduzione bidirezionale) per dedurre qualcosa di forte.

9.7 Amico con problema EXP-completo trovato anche in NP

Se L è **EXP-completo** ed è anche in NP, allora: ogni problema in EXP si riduce polinomialmente a L (per completezza) $\Rightarrow$ se $L \in \text{NP}$, allora $\text{EXP} \subseteq \text{NP}$ (le riduzioni polinomiali trasportano l'appartenenza a NP) $\Rightarrow$ se fosse anche $\text{NP} = \text{P}$, avresti $\text{EXP} \subseteq \text{P}$, assurdo per il Time Hierarchy Theorem ($\text{P} \subsetneq \text{EXP}$, certo). Quindi deve essere $\text{NP} \neq \text{P}$, cioè il tuo amico (se ha ragione sull'esistenza di L) avrebbe dimostrato $\text{P} \neq \text{NP}$.

Confronto tra i due pattern (9.6 vs 9.7): parlare di un sovrainsieme facile di un problema hard non dice nulla; parlare di un problema **completo per una classe** (che

rappresenta il caso più difficile di quella classe) può dare conclusioni forti se lo trovi anche in una classe più piccola.

10. Checklist finale per rispondere ai quiz

1. **La domanda riguarda il comportamento generico di una TM** (accetta/rifiuta/termina su un dato input)? → sospetta **indecidibilità** (Teorema di Rice), quasi certamente anche “not in P”.
2. **C'è un parametro (k, numero variabili...) fissato a una costante nella definizione del problema**, non libero nell'istanza? → sospetta **P** invece di NP-completo.
3. **La domanda confronta due funzioni con esponenti “strani”** (n^n , 2^n , fattoriali)? → calcola il **rapporto** o usa **Stirling**, guarda prima come cresce l'esponente stesso.
4. **La domanda chiede relazioni tra classi (P, NP, EXP, NPC...)?** → ricorda che solo le separazioni **verso EXP/EXPSPACE** (hierarchy theorem) sono certe; tutto il resto con NP/NPC è congetturato.
5. **La domanda parla di “complemento” di un problema?** → guarda la classe del problema originale: complemento di P resta P; complemento di NP-completo è coNP-completo (non “NP-completo”).
6. **La domanda chiede di dimostrare $L \in NP$ (o FP)?** → usa sempre lo schema fisso: certificato di dimensione polinomiale + verificatore con le 4 condizioni di polinomialità.
7. **La domanda chiede di dimostrare NP-completezza?** → NP (schema sopra) + riduzione **bidirezionale** da un problema NP-completo noto.
8. **La domanda riguarda PAC-learning/VC-dimension di una forma geometrica semplice?** → quasi sempre efficiently PAC-learnable con VC-dimension piccola; sospetta difficoltà solo se la struttura è combinatoria (tipo DNF/CNF con molte variabili libere).